

MIDTERM (SHORT NOTES)

2024 - 2025

CS101 [Definitions]

Contact Us: 03254755408

Module 1: Introduction to Computer Science

- ★ **Computer Science** is the study of computation, automation, and information.
- ★ **Hardware** refers to the physical components of a computer system.
- ★ **Software** is a set of instructions that tell the computer what to do.
- ★ **System Software** manages hardware and creates the environment for application software.
- ★ **Application Software** helps users perform specific tasks like word processing.
- ★ **Programming** is the process of creating software using a programming language.

Module 2: Breadth First Learning

- ★ **Search Engine Usage** teaches how to retrieve information effectively online.
- ★ **History of Computing** highlights the development of computers from early tools to modern machines.
- ★ **Data Storage** involves methods to save digital information, such as RAM and hard drives.
- ★ **Data Manipulation** refers to processing and handling data using a computer.
- ★ **Operating System** is the software that controls and coordinates hardware and applications.
- ★ **Networking** is the communication between two or more computers.

Module 3: Search Engines

- ★ **Query** is a keyword or phrase entered into a search engine.
 - ★ **Search Engine** is a tool that locates information on the internet.
 - ★ **Indexing** is how search engines organize and store web page data.
 - ★ **Search Algorithm** determines how search results are ranked and displayed.
 - ★ **Ranking** means placing search results based on relevance.
 - ★ **Crawler** is a bot that scans web content to build the search index.
-

Module 4: Searching Tricks

- ★ **Weather Searching** allows users to check the weather by location directly on search engines.
 - ★ **Google Calculator** lets users perform math using search bar input.
 - ★ **Currency Converter** provides exchange rates and conversions.
 - ★ **Time Conversion** can show current time in any global city.
 - ★ **Definition Search** returns word meanings using “define:” operator.
 - ★ **Search Filters** refine results by date, type, or region.
-

Module 5: Search Operators (1)

- ★ **Social Media Search** allows finding social content like tweets or posts.
 - ★ **Price Search** finds product prices directly through search input.
 - ★ **Hashtag Search** locates social content tagged with specific hashtags.
 - ★ **Exclude Words** uses “-” to remove unwanted terms from results.
 - ★ **Exact Match** searches phrases in quotes to return precise results.
 - ★ **Wildcard Search** uses asterisks (*) to represent unknown words in a query.
-

Module 6: Search Operators (2)

- ★ **Range Search** helps find items between two numbers using “..” (e.g., 2010..2020).
 - ★ **Boolean Operators** like AND, OR, NOT refine search results logically.
 - ★ **Site Search** restricts results to a specific domain using “site:”.
 - ★ **Related Search** finds sites similar to the one you specify using “related:”.
 - ★ **Cached Search** accesses saved versions of web pages.
 - ★ **Define Operator** provides definitions from various dictionaries.
-

Module 7: Search Operators (3)

- ★ **Stocks Operator** displays real-time stock data.
 - ★ **Map Operator** shows location maps for the given place.
 - ★ **Movie Operator** retrieves movie showtimes and information.
 - ★ **Food Comparison** helps compare nutritional values of foods.
 - ★ **Tilt/Easter Eggs** gives funny or unique search results.
 - ★ **Image Search** finds pictures matching your keywords.
-

Module 8: Advanced Search Operators

- ★ **intitle:** restricts results to pages with certain words in the title.
 - ★ **allintitle:** ensures all words appear in the title.
 - ★ **inurl:** finds pages with specific words in the URL.
 - ★ **allinurl:** all listed words must be in the page's URL.
 - ★ **intext:** shows pages where words appear in the content body.
 - ★ **Proximity Search** looks for terms close together on a page.
-

Module 9: What We Should Not Search

- ★ **Avoid Ads** by using ad blockers and avoiding vague terms.
 - ★ **Dangerous Searches** can lead to scams or malware.
 - ★ **Cybersecurity Threats** arise from clicking unsafe links.
 - ★ **Unpleasant Results** can come from sensitive or explicit searches.
 - ★ **Illegal Content** should be completely avoided.
 - ★ **Misinformation** can spread from unreliable sources.
-

Module 10: Roots of Computing

- ★ **Abacus** is one of the earliest computing devices used for calculations.
 - ★ **Gear Technology** helped in building mechanical computing devices.
 - ★ **Punch Cards** were used for input in early computers.
 - ★ **ENIAC** was one of the first electronic general-purpose computers.
 - ★ **Vacuum Tubes** were used in early computers to control current.
 - ★ **Computing Evolution** shows how computers developed from mechanical to electronic.
-

Module 11: Bits

- ★ **Bit** is the smallest unit of data in computing, representing a 0 or 1.
 - ★ **Bit Pattern** refers to a combination of bits like 1010 used to represent information.
 - ★ **Byte** consists of 8 bits.
 - ★ **Kilobyte (KB)** equals 1024 bytes.
 - ★ **Megabyte (MB)** equals 1024 KB.
 - ★ **Gigabyte (GB)** equals 1024 MB.
-

Module 12: Boolean Operations

- ★ **AND Operation** returns true only if both inputs are true.
 - ★ **OR Operation** returns true if at least one input is true.
 - ★ **XOR Operation** returns true if inputs are different.
 - ★ **NOT Operation** reverses the input value.
 - ★ **Boolean Logic** is essential for computer decision-making.
 - ★ **Truth Table** lists all possible input/output combinations for logic gates.
-

Module 13: Hexadecimal Notation

- ★ **Hexadecimal** is base-16 number system using digits 0-9 and letters A-F.
 - ★ **Why Hexadecimal?** It shortens long binary sequences.
 - ★ **Conversion** from binary to hexadecimal is done by grouping 4-bit chunks.
 - ★ **Example:** Binary 1111 = Hex F.
 - ★ **Each hex digit** represents 4 binary bits.
 - ★ **Used in** memory addressing and color coding.
-

Module 14: Storing a Bit

- ★ **Main Memory** stores data temporarily for quick access.
 - ★ **Byte Organization** involves arranging memory in 8-bit units.
 - ★ **Memory Address** identifies each byte's location.
 - ★ **RAM (Random Access Memory)** allows data to be read/written quickly.
 - ★ **DRAM** is a type of RAM that needs refreshing.
 - ★ **SDRAM** is synchronous DRAM working with the system clock.
-

Module 15: Magnetic Systems

- ★ **Mass Storage** includes hard drives, tapes, and SSDs.
- ★ **Zoned-bit Recording** splits the disk into zones for storage efficiency.

- ★ **Seek Time** is how fast the head reaches the track.
 - ★ **Rotation Delay** is the time to spin data under the head.
 - ★ **Access Time** is seek time + rotation delay.
 - ★ **Transfer Rate** is how fast data is read or written.
-

Module 16: Optical Systems

- ★ **Compact Disk (CD)** stores data with a laser-readable surface.
 - ★ **DVD** holds more data than CDs, used for video storage.
 - ★ **Blu-ray Disk (BD)** stores high-definition content.
 - ★ **Single Spiral Track** stores data in CDs and DVDs.
 - ★ **Laser Reading** is used instead of magnetic heads.
 - ★ **Slower Access** compared to magnetic disks due to linear reading.
-

Module 17: Flash Drives

- ★ **Flash Drives** use flash memory for storage.
 - ★ **No Moving Parts** means faster access and more durability.
 - ★ **SSDs** are faster and more reliable than traditional hard drives.
 - ★ **Secure Digital (SD) Cards** are small, portable storage media.
 - ★ **Flash Memory** retains data without power.
 - ★ **Replaces** both optical and magnetic storage in many uses.
-

Module 18: Representing Text

- ★ **ASCII** assigns numbers to letters and symbols.
 - ★ **Limitations of ASCII** include limited language support.
 - ★ **Unicode** represents text for all world languages.
 - ★ **UTF-8** is a Unicode encoding that's backward compatible with ASCII.
 - ★ **Text as Code** converts characters into digital form.
 - ★ **Example:** 'A' = 65 in ASCII.
-

Module 19: Representing Numeric Values

- ★ **Binary Notation** uses 0s and 1s to represent numbers.
- ★ **Unicode for Numbers** is inefficient and incorrect.
- ★ **Decimal to Binary** conversion is essential in computing.
- ★ **Variations** include 2's complement, floating-point.

- ★ **Numeric Storage** allows mathematical operations in computers.
 - ★ **Efficient Representation** ensures accuracy and performance.
-

Module 20: Representing Images

- ★ **Pixel** is the smallest unit of an image.
 - ★ **Bitmap Encoding** maps each pixel to a color value.
 - ★ **Shades Handling** uses grayscale for depth.
 - ★ **Color Encoding** uses RGB combinations.
 - ★ **Image Scaling** adjusts resolution without distorting quality.
 - ★ **Scalable Fonts** use vector graphics for crisp resizing.
-

Module 21: Representing Sound

- ★ **Amplitude** refers to the loudness or strength of a sound wave.
 - ★ **Sound Encoding** captures sound by sampling at regular intervals.
 - ★ **Sampling Rate** is how often sound is measured per second (e.g., 8000 samples/sec).
 - ★ **Sound Storage** uses numerical values to digitally represent sound.
 - ★ **Communication of Sound** sends audio over networks using digitized signals.
 - ★ **MIDI (Musical Instrument Digital Interface)** is an alternative digital format for music.
-

Module 22: Binary Notation

- ★ **Binary Notation** uses base-2 with only digits 0 and 1.
 - ★ **Power Method** calculates binary value by powers of 2.
 - ★ **Binary Representation** is used to store numeric values in computers.
 - ★ **Bit Significance**: Each binary digit has a positional value.
 - ★ **Conversion Algorithm** helps turn decimal numbers into binary.
 - ★ **Use in Computing** is foundational to digital systems.
-

Module 23: Binary Addition

- ★ **Binary Addition** follows rules: $0+0=0$, $1+0=1$, $1+1=10$.
- ★ **Carry in Binary** occurs when sum exceeds 1 (just like base-10).
- ★ **Bit-wise Addition** adds corresponding bits with carry.
- ★ **Overflow** happens when result exceeds bit storage.

- ★ **Two-bit Example:** $01 + 01 = 10$.
 - ★ **Importance:** Binary addition underlies computer arithmetic.
-

Module 24: Fraction in Binary

- ★ **Radix Point** is the binary equivalent of a decimal point.
 - ★ **Binary Fractions** are formed by negative powers of 2.
 - ★ **Conversion** requires multiplying fraction by 2 repeatedly.
 - ★ **Addition of Fractions** is similar to whole binary numbers.
 - ★ **Precision Limitation** means not all decimals can be exact in binary.
 - ★ **Applications** include scientific and financial computing.
-

Module 25: 2's Complement Notation

- ★ **2's Complement** is the standard for storing signed integers.
 - ★ **Sign Bit** indicates positive (0) or negative (1) number.
 - ★ **Positive/Negative Conversion** uses a special binary rule.
 - ★ **Binary Addition** works seamlessly with 2's complement.
 - ★ **Overflow Detection** checks the sign bits after addition.
 - ★ **Example:** $+7 = 0111$; $-7 = 1001$ (in 4-bit representation).
-

Module 26: Excess Notation

- ★ **Excess Notation** shifts values by a fixed amount (bias).
 - ★ **Bias Value** is added to actual value to store.
 - ★ **Excess-4 Notation** shifts values by 4.
 - ★ **Comparisons** become easier as numbers remain unsigned.
 - ★ **Example:** Value 0 becomes 4 in excess-4.
 - ★ **Use Case:** Often used in floating-point exponents.
-

Module 27: Floating Point Notation

- ★ **Floating Point** represents real numbers using mantissa and exponent.
- ★ **Radix** (base) is commonly 2 in computers.
- ★ **Mantissa** stores the digits.
- ★ **Exponent** determines placement of radix point.
- ★ **Scientific Notation** inspired floating point design.
- ★ **Used for** large or fractional number calculations.

Module 28: Truncation Errors in Floating Point

- ★ **Truncation Error** occurs when digits are cut off.
 - ★ **Precision Loss** means not all numbers are accurately stored.
 - ★ **Primitive Methods** include rounding or cutting off decimals.
 - ★ **Intelligent Processing** adjusts computations to minimize errors.
 - ★ **Cumulative Errors** can affect calculations in large programs.
 - ★ **Important in** scientific computing and simulations.
-

Module 29: Data Compression (Generic Techniques)

- ★ **Data Compression** reduces file size for storage or transmission.
 - ★ **Lossless Compression** retains original data perfectly.
 - ★ **Lossy Compression** discards some data to save space.
 - ★ **Huffman Coding** is a lossless technique using shorter codes.
 - ★ **Run-Length Encoding** stores repeated values as a single value and count.
 - ★ **Compression Ratio** measures effectiveness of compression.
-

Module 30: Data Compression (Compressing Images)

- ★ **Image Compression** reduces size of picture files.
 - ★ **JPEG** is a common lossy format.
 - ★ **GIF/PNG** are lossless image formats.
 - ★ **Color Reduction** simplifies image data for smaller files.
 - ★ **Transform Techniques** like DCT are used in JPEG.
 - ★ **Trade-off** is between image quality and file size.
-

Module 31: Compressing Audio and Videos

- ★ **Audio Compression** reduces the size of sound files while maintaining quality.
 - ★ **Video Compression** reduces file size by removing unnecessary frames or sounds.
 - ★ **MP3** is a widely used audio compression format.
 - ★ **MPEG** is a standard for video compression.
 - ★ **Streaming** allows playing audio/video without full download.
 - ★ **Codecs** are programs that compress and decompress multimedia.
-

Module 32: CPU Basic

- ★ **CPU (Central Processing Unit)** is the brain of the computer.
 - ★ **Control Unit** directs operations inside the CPU.
 - ★ **ALU (Arithmetic Logic Unit)** performs calculations and logic.
 - ★ **Registers** are small memory units within the CPU.
 - ★ **Fetch-Decode-Execute** is the instruction processing cycle.
 - ★ **Instruction Set** is the group of commands a CPU can execute.
-

Module 33: Stored Program

- ★ **Stored Program Concept** means instructions are stored in memory.
 - ★ **Memory** holds both data and instructions.
 - ★ **Program Counter (PC)** keeps track of the next instruction.
 - ★ **Instruction Register (IR)** holds the current instruction.
 - ★ **Von Neumann Architecture** uses the stored program model.
 - ★ **Benefit:** Flexible execution of any software.
-

Module 34: CPU Architecture Philosophies

- ★ **CISC (Complex Instruction Set Computing)** uses complex, multi-step instructions.
 - ★ **RISC (Reduced Instruction Set Computing)** uses simple, fast instructions.
 - ★ **Micro-operations** are small actions performed by the CPU.
 - ★ **Instruction Cycle** involves multiple internal steps.
 - ★ **Efficiency** differs between CISC and RISC systems.
 - ★ **Trade-off:** Complexity vs. Speed.
-

Module 35: Machine Instruction Categories

- ★ **Data Transfer Instructions** move data between locations.
 - ★ **Arithmetic Instructions** perform calculations.
 - ★ **Logical Instructions** handle comparisons and conditions.
 - ★ **Control Instructions** manage program flow (e.g., jumps).
 - ★ **Instruction Groups** are categorized by function.
 - ★ **Execution** varies depending on the instruction type.
-

Module 36: Program Execution

- ★ **Execution Cycle** involves fetching, decoding, and running instructions.
 - ★ **PC (Program Counter)** helps track program flow.
 - ★ **IR (Instruction Register)** decodes the current command.
 - ★ **ALU** carries out arithmetic/logic functions.
 - ★ **Control Signals** coordinate CPU components.
 - ★ **Status Flags** indicate outcomes (e.g., zero, carry).
-

Module 37: Program Execution Example

- ★ **Step-by-step Execution** helps understand instruction cycles.
 - ★ **Example Programs** demonstrate CPU tasks in action.
 - ★ **Instruction Flow** follows fetch-decode-execute.
 - ★ **Register Use** is shown clearly in examples.
 - ★ **Memory Access** demonstrates data movement.
 - ★ **Practical Understanding** improves learning outcomes.
-

Module 38: Logic Operators

- ★ **AND Operator** returns true if both inputs are true.
 - ★ **OR Operator** returns true if at least one input is true.
 - ★ **NOT Operator** reverses the input value.
 - ★ **XOR Operator** returns true only if inputs differ.
 - ★ **Boolean Expressions** are built from logic operators.
 - ★ **Use in Programming** allows conditional operations.
-

Module 39: Rotation and Shift

- ★ **Bit Rotation** moves bits around in a circular manner.
 - ★ **Logical Shift** moves bits and fills empty spots with zeros.
 - ★ **Arithmetic Shift** keeps the sign bit while shifting.
 - ★ **Use in Cryptography** and bit-level manipulation.
 - ★ **Shifting Right/Left** affects value interpretation.
 - ★ **Important for** optimizing performance in coding.
-

Module 40: Arithmetic Operators

- ★ **Addition Operator (+)** adds two values.
- ★ **Subtraction Operator (-)** subtracts one from another.

- ★ **Multiplication Operator (*)** multiplies values.
 - ★ **Division Operator (/)** divides numbers.
 - ★ **Modulo Operator (%)** gives the remainder.
 - ★ **Used in Programs** to perform calculations and logic.
-

Module 41: Role of Controller

- ★ **Controller** is a component that manages a specific hardware device.
 - ★ **Bus Communication** allows data to travel between CPU, memory, and controllers.
 - ★ **Synchronization** is needed to coordinate CPU and controller actions.
 - ★ **Controller's Role** includes managing I/O devices and buffering data.
 - ★ **Performance Boost** is achieved when controllers handle operations independently.
 - ★ **Von Neumann Bottleneck** is the CPU and controller competition over the bus.
-

Module 42: Direct Memory Access and Handshaking

- ★ **Direct Memory Access (DMA)** allows controllers to access memory directly.
 - ★ **Handshaking** is the process of coordinating communication between devices.
 - ★ **Bus Utilization** improves with DMA as CPU handles other tasks.
 - ★ **Data Transfer** is managed by controller during CPU idle time.
 - ★ **Parallel Activity** allows simultaneous CPU execution and memory I/O.
 - ★ **Von Neumann Architecture** faces a bottleneck due to shared communication lines.
-

Module 43: Communication Media and Communication Rates

- ★ **Parallel Communication** transmits multiple bits at once on separate lines.
 - ★ **Serial Communication** sends data bit-by-bit on a single path.
 - ★ **Modem** converts digital signals to analog and vice versa.
 - ★ **USB & FireWire** provide high-speed short-distance communication.
 - ★ **DSL & Cable** support long-distance, fast internet.
 - ★ **Data Rate Units** include Kbps, Mbps, and Gbps.
-

Module 44: Pipelining

- ★ **Pipelining** allows overlapping of CPU instruction steps.
- ★ **Throughput** increases by processing multiple instructions at once.
- ★ **Miniaturization Limit** affects speed due to physical distances.

- ★ **Execution Stages** are separated to allow parallel progress.
 - ★ **Fetch While Execute** improves performance.
 - ★ **Jump Instructions** disrupt pipelining efficiency.
-

Module 45: Operating Systems - History

- ★ **Operating Systems (OS)** manage hardware and software resources.
 - ★ **Batch Processing** was an early OS feature.
 - ★ **Time Sharing** allows multiple users to interact with a computer.
 - ★ **Multiprogramming** enables running several programs concurrently.
 - ★ **Evolution of OS** tracks progress from punch cards to GUIs.
 - ★ **Mainframe OS** served business and scientific needs initially.
-

Module 46: Operating Systems - Basic Concepts (I)

- ★ **Resource Manager** is a key role of the OS.
 - ★ **Process Management** handles execution of applications.
 - ★ **Memory Management** allocates and tracks RAM use.
 - ★ **File System** organizes data storage and retrieval.
 - ★ **Device Management** controls input/output hardware.
 - ★ **User Interface** is how users interact with the OS.
-

Module 47: Operating Systems - Basic Concepts (II)

- ★ **Multitasking** allows switching between active tasks.
 - ★ **Spooling** stores jobs in buffer for orderly processing.
 - ★ **Interrupts** signal the CPU to stop and handle events.
 - ★ **Job Scheduling** decides the execution order of processes.
 - ★ **Buffering** temporarily holds data during transfer.
 - ★ **Kernel** is the core of the OS.
-

Module 48: Operating Systems - Software Classification

- ★ **System Software** includes OS and utility tools.
- ★ **Application Software** performs specific user tasks.
- ★ **Translators** convert programming languages (e.g., compilers).
- ★ **Device Drivers** enable OS to control hardware.

- ★ **Middleware** connects software and services.
 - ★ **Firmware** is software embedded into hardware.
-

Module 49: Operating Systems - Components (I)

- ★ **Memory Manager** handles allocation and protection of RAM.
 - ★ **Processor Manager** assigns tasks to the CPU.
 - ★ **Device Manager** interacts with hardware devices.
 - ★ **File Manager** organizes and accesses file systems.
 - ★ **User Interface** supports command-line or GUI interaction.
 - ★ **System Calls** provide access to OS services.
-

Module 50: Operating Systems - Components (II)

- ★ **Process Table** keeps track of active processes.
 - ★ **Ready Queue** lists processes waiting for CPU time.
 - ★ **I/O Queue** holds processes needing device access.
 - ★ **Interrupt Handlers** respond to hardware signals.
 - ★ **Schedulers** decide which process runs next.
 - ★ **Security Components** guard resources and data integrity.
-

Module 51: Process of Booting

- ★ **Booting** is the process of starting a computer and loading the operating system.
 - ★ **BIOS** initializes hardware during the startup.
 - ★ **Boot Loader** loads the operating system into memory.
 - ★ **POST (Power-On Self-Test)** checks system components.
 - ★ **Kernel** is loaded into memory to control system functions.
 - ★ **Cold Boot vs Warm Boot**: Cold is from power-off state; warm is restart.
-

Module 52: Process and Its Administration

- ★ **Process** is a program in execution.
- ★ **Process Control Block (PCB)** stores process details.
- ★ **Process States** include ready, running, waiting, and terminated.
- ★ **Scheduling** decides which process runs next.
- ★ **Process ID (PID)** uniquely identifies each process.
- ★ **State Transitions** are managed by the OS.

Module 53: Process Administration (Cont.)

- ★ **Context Switching** saves and restores process data.
 - ★ **Ready Queue** holds processes waiting for CPU.
 - ★ **I/O Queue** stores processes waiting for I/O.
 - ★ **Interrupts** trigger context switches.
 - ★ **Multitasking** simulates parallelism using fast switching.
 - ★ **Priority Scheduling** runs higher-priority processes first.
-

Module 54: Handling Competition between Processes

- ★ **Resource Allocation** distributes CPU, memory, I/O.
 - ★ **File Manager** manages file access and storage.
 - ★ **Scheduler** handles process timing.
 - ★ **Dispatcher** assigns time slices.
 - ★ **Deadlock** can occur with resource contention.
 - ★ **Algorithms** must prevent resource conflicts.
-

Module 55: Semaphores

- ★ **Semaphore** is a variable used to control access to resources.
 - ★ **Binary Semaphore** has values 0 and 1.
 - ★ **Wait and Signal** are operations to modify semaphores.
 - ★ **Mutual Exclusion** prevents simultaneous resource use.
 - ★ **Race Condition** happens when two processes access data at once.
 - ★ **Critical Section** is the code part needing exclusive access.
-

Module 56: Deadlock

- ★ **Deadlock** is a situation where processes wait on each other forever.
 - ★ **Circular Wait** is a condition for deadlock.
 - ★ **Hold and Wait** causes blocking.
 - ★ **No Preemption** makes resources non-recoverable.
 - ★ **Deadlock Prevention** avoids necessary conditions.
 - ★ **Deadlock Recovery** reboots or terminates processes.
-

Module 57: Security Attacks from Outside

- ★ **Malware** includes viruses, worms, and spyware.
 - ★ **Phishing** tricks users into revealing data.
 - ★ **Brute Force Attacks** try all password combinations.
 - ★ **Firewalls** block unauthorized access.
 - ★ **Antivirus Software** detects and removes threats.
 - ★ **Encryption** protects data from interception.
-

Module 58: Security Attacks from Inside

- ★ **Internal Threats** come from employees or users.
 - ★ **Privilege Abuse** is misuse of access rights.
 - ★ **Backdoors** allow secret access.
 - ★ **Data Theft** is stealing sensitive info.
 - ★ **Audit Trails** track user activities.
 - ★ **Access Control** restricts data availability.
-

Module 59: Network Classification

- ★ **LAN (Local Area Network)** connects nearby devices.
 - ★ **WAN (Wide Area Network)** spans large geographic areas.
 - ★ **MAN (Metropolitan Area Network)** covers a city.
 - ★ **Topology** refers to physical layout (e.g., star, ring).
 - ★ **Peer-to-Peer** networks share resources directly.
 - ★ **Client-Server** models centralize resources.
-

Module 60: Protocols

- ★ **Protocol** is a set of rules for data exchange.
 - ★ **TCP/IP** is the standard internet protocol suite.
 - ★ **HTTP/HTTPS** handles web page communication.
 - ★ **FTP** is used for file transfers.
 - ★ **IP Addressing** identifies devices on networks.
 - ★ **Network Layers** organize communication tasks.
-

Module 61: Combining Networks

- ★ **Network Interconnection** links multiple networks into one.
 - ★ **Router** forwards data between networks.
 - ★ **Bridge** connects and filters traffic between segments.
 - ★ **Gateway** acts as a translator between different protocols.
 - ★ **Internet** is the largest interconnected network.
 - ★ **Network Expansion** enhances communication and resource sharing.
-

Module 62: Methods of Process Communication

- ★ **Process Communication** enables programs to exchange data.
 - ★ **Client/Server Model** separates requester and provider.
 - ★ **Peer-to-Peer Model** allows equal roles for communication.
 - ★ **Sockets** are endpoints for sending/receiving data.
 - ★ **Ports** identify specific applications or services.
 - ★ **Inter-process Communication (IPC)** includes shared memory and messaging.
-

Module 63: Distributed Systems

- ★ **Distributed System** is a group of computers working together.
 - ★ **Transparency** hides the complexity from users.
 - ★ **Scalability** supports growth by adding resources.
 - ★ **Fault Tolerance** allows system function despite failures.
 - ★ **Resource Sharing** allows multiple users to use data/devices.
 - ★ **Concurrency** enables simultaneous task execution.
-

Module 64: Internet Architecture

- ★ **Tiered Architecture** organizes the internet into layers.
 - ★ **Access ISPs** connect users to larger networks.
 - ★ **Regional ISPs** provide broader internet services.
 - ★ **Backbone Networks** form the core of the internet.
 - ★ **Protocols** define communication rules.
 - ★ **Hierarchical Structure** maintains efficiency and control.
-

Module 65: Internet Addressing

- ★ **IP Address** is a unique identifier for each device.
- ★ **IPv4** uses 32-bit addresses (e.g., 192.168.0.1).

- ★ **IPv6** uses 128-bit addresses for larger space.
 - ★ **Subnetting** divides networks into smaller parts.
 - ★ **Public IP** is globally accessible; **Private IP** is local.
 - ★ **DNS (Domain Name System)** maps names to IPs.
-

Module 66: Internet Applications

- ★ **Email** allows sending text and attachments over the internet.
 - ★ **Web Browsing** retrieves and displays web pages.
 - ★ **File Transfer (FTP)** moves files between computers.
 - ★ **VoIP (Voice over IP)** enables calls via the internet.
 - ★ **Remote Login** accesses systems from afar.
 - ★ **Streaming** delivers continuous media like video/audio.
-

Module 67: Internet Applications - Email

- ★ **Email Client** software like Outlook manages emails.
 - ★ **SMTP (Simple Mail Transfer Protocol)** sends messages.
 - ★ **POP/IMAP** receive emails from servers.
 - ★ **Email Header** contains sender, subject, and time.
 - ★ **Attachments** allow sending files with messages.
 - ★ **Spam Filters** block unwanted mail.
-

Module 68: VoIP

- ★ **VoIP** allows voice communication via the internet.
 - ★ **Packet Switching** sends voice as data packets.
 - ★ **Latency** is the delay during transmission.
 - ★ **Jitter** is the variation in packet arrival time.
 - ★ **VoIP Clients** include Skype, Zoom, etc.
 - ★ **Cost Efficiency** is a major VoIP advantage.
-

Module 69: Internet Multimedia Streaming

- ★ **Streaming** plays media in real-time as it downloads.
- ★ **Buffering** preloads media for smooth playback.
- ★ **Compression** reduces file size for faster delivery.
- ★ **Video Codecs** like H.264 encode media.

- ★ **Live Streaming** broadcasts in real-time.
 - ★ **Bitrate** affects quality and data usage.
-

Module 70: World Wide Web

- ★ **WWW** is a system of interlinked web pages.
 - ★ **HTML** structures the content.
 - ★ **HTTP** is the protocol used for web data transfer.
 - ★ **Web Server** hosts and serves website content.
 - ★ **Web Browser** requests and displays websites.
 - ★ **Hyperlinks** allow navigation between documents.
-

Module 71: Web Implementations

- ★ **URL (Uniform Resource Locator)** specifies the address of web resources.
 - ★ **HTTP** is the protocol for data communication on the web.
 - ★ **Anchor Tag (<a>)** is used to create hyperlinks.
 - ★ **Hyperlink** connects one web page to another.
 - ★ **Web Browser** is a program to view websites (e.g., Chrome).
 - ★ **Client-Server Model** defines the interaction between user and web server.
-

Module 72: HTML (I)

- ★ **HTML (HyperText Markup Language)** is the standard for creating web pages.
 - ★ **<html> Tag** begins and ends an HTML document.
 - ★ **<body> Tag** contains all visible content.
 - ★ **<p> Tag** is used to define paragraphs.
 - ★ ** Tag** makes the text bold.
 - ★ **Tags** are keywords surrounded by angle brackets.
-

Module 73: HTML (II)

- ★ **<a> Tag** defines a hyperlink.
- ★ **href Attribute** specifies the link destination.
- ★ ** Tag** embeds images into a webpage.
- ★ **src Attribute** provides the image file location.
- ★ **Alt Attribute** offers alternative text for images.
- ★ **HTML Layout** uses tags to structure content visually.

Module 74: More on HTML

- ★ **<form> Tag** creates a user input form.
 - ★ **<input> Tag** collects text, buttons, checkboxes, etc.
 - ★ **<table> Tag** organizes content into rows and columns.
 - ★ **, , Tags** create unordered and ordered lists.
 - ★ **
 Tag** inserts a line break.
 - ★ **HTML Attributes** define extra information about elements.
-

Module 75: XML

- ★ **XML (eXtensible Markup Language)** is used for data storage and transfer.
 - ★ **User-defined Tags** allow flexible data formatting.
 - ★ **Well-Formed Document** follows XML syntax strictly.
 - ★ **Element** holds data between opening and closing tags.
 - ★ **Attribute** provides additional info within an element tag.
 - ★ **Used in** web services, config files, and data exchange.
-

Module 76: Client-Side and Server-Side

- ★ **Client-Side Scripting** runs in the browser (e.g., JavaScript).
 - ★ **Server-Side Scripting** runs on the web server (e.g., PHP).
 - ★ **Validation** checks data on client/server before submission.
 - ★ **Dynamic Pages** are generated based on user interaction.
 - ★ **Security** is stronger on the server side.
 - ★ **Efficiency** improves when combining both approaches.
-

Module 77: Layered Internet Software (I)

- ★ **Layered Architecture** separates concerns in communication systems.
 - ★ **Application Layer** handles user-level interactions (e.g., HTTP).
 - ★ **Transport Layer** manages data delivery (e.g., TCP).
 - ★ **Network Layer** routes data across networks (e.g., IP).
 - ★ **Data Link Layer** manages local data transmission.
 - ★ **Each Layer** serves a specific function in communication.
-

Module 78: Layered Internet Software (II)

- ★ **OSI Model** consists of 7 layers, each with a unique role.
 - ★ **TCP/IP Model** is a 4-layer model used on the internet.
 - ★ **Encapsulation** wraps data with protocol headers.
 - ★ **Addressing** assigns unique IDs to devices.
 - ★ **Ports** identify services running on a device.
 - ★ **Protocols** define data communication rules.
-

Module 79: Layered Internet Software (III)

- ★ **Data Transmission** passes through multiple software layers.
 - ★ **Header** contains metadata like address and type.
 - ★ **Packet** is a small block of data sent over networks.
 - ★ **Flow Control** prevents sender from overwhelming receiver.
 - ★ **Error Detection** identifies data corruption.
 - ★ **Layer Responsibilities** ensure smooth end-to-end communication.
-

Module 80: TCP/IP Protocol Suite

- ★ **TCP (Transmission Control Protocol)** provides reliable data transfer.
 - ★ **IP (Internet Protocol)** handles addressing and routing.
 - ★ **UDP (User Datagram Protocol)** is fast but unreliable.
 - ★ **IPv4 vs. IPv6** refers to 32-bit vs. 128-bit addressing.
 - ★ **Socket** is a communication endpoint for data exchange.
 - ★ **Protocol Stack** is a combination of TCP/IP layers used in communication.
-

Module 81: Security (Forms of Attacks)

- ★ **Malware** is malicious software like viruses, worms, or Trojans.
 - ★ **Denial of Service (DoS)** attacks overload systems to shut them down.
 - ★ **Phishing** tricks users into giving up personal data.
 - ★ **Spoofing** involves impersonating another device or user.
 - ★ **Sniffing** intercepts and reads data packets on a network.
 - ★ **SQL Injection** injects malicious code into database queries.
-

Module 82: Protection and Cures

- ★ **Firewall** controls incoming and outgoing network traffic.
 - ★ **Antivirus Software** detects and removes malicious programs.
 - ★ **Proxy Server** acts as a mediator between client and server for added protection.
 - ★ **Encryption** scrambles data to prevent unauthorized access.
 - ★ **Authentication** verifies the identity of users or devices.
 - ★ **Access Control** limits user access to system resources.
-

Module 83: Encryption

- ★ **Encryption** converts plain text into unreadable form.
 - ★ **Decryption** reverses encryption to get original data.
 - ★ **Symmetric Encryption** uses one key for both encryption and decryption.
 - ★ **Asymmetric Encryption** uses a public and a private key.
 - ★ **Digital Certificate** proves the identity of a website or user.
 - ★ **SSL/TLS** are secure protocols used for encrypted web communication.
-

Module 84: Legal Approaches to Network Security

- ★ **Computer Fraud and Abuse Act** is U.S. law against cybercrime.
 - ★ **Cyber Law** defines the legal use of digital data and devices.
 - ★ **Privacy Laws** protect user data from unauthorized access.
 - ★ **Intellectual Property Laws** defend software and digital creations.
 - ★ **Digital Evidence** is used in legal investigations involving computers.
 - ★ **Jurisdiction Issues** arise when cybercrimes cross borders.
-

Module 85: Algorithm - An Informal Review

- ★ **Algorithm** is a set of steps to solve a problem.
 - ★ **Fetch-Decode-Execute Cycle** is the core of executing instructions.
 - ★ **Input-Process-Output** is the basic computing model.
 - ★ **Termination** means the algorithm eventually ends.
 - ★ **Execution Steps** must be clear and followable.
 - ★ **Algorithm Use** spans decision-making to imagination simulation.
-

Module 86: Formal Definition of Algorithm

- ★ **Formal Algorithm** must be ordered, unambiguous, and executable.
- ★ **Order vs Sequence** emphasizes structure over strict step order.

- ★ **Unambiguous** means each step has only one clear interpretation.
 - ★ **Executable** ensures a computer can perform all steps.
 - ★ **Effective** steps must lead to a result or action.
 - ★ **Termination** implies the process ends with an outcome.
-

Module 87: Abstract Nature of Algorithms

- ★ **Abstract Concept** means algorithms are ideas, not their code.
 - ★ **Representation** can be text, formulas, or flowcharts.
 - ★ **Same Algorithm** can have multiple forms.
 - ★ **Communication** requires clarity depending on the audience.
 - ★ **Level of Detail** can differ by user expertise.
 - ★ **Understanding** the difference improves flexibility in problem-solving.
-

Module 88: Algorithm Representation (Primitives)

- ★ **Primitives** are basic operations understood by the system.
 - ★ **Examples** include math operations, comparisons, assignments.
 - ★ **Atomic Actions** mean the operation can't be broken down further.
 - ★ **Building Blocks** form complex algorithms using simple actions.
 - ★ **Flow of Control** is determined by combining primitives.
 - ★ **Standardization** ensures universal understanding of algorithm steps.
-

Module 89: Algorithm Representation (Pseudocode)

- ★ **Pseudocode** is a way to describe algorithms informally.
 - ★ **Not a Programming Language** but follows logical rules.
 - ★ **Human-Readable** for both technical and non-technical users.
 - ★ **Control Structures** like IF, ELSE, WHILE are used.
 - ★ **Helps in Planning** before actual coding.
 - ★ **Flexibility** allows adaptation across programming languages.
-

Module 90: Pseudocode - While Structure

- ★ **While Loop** executes as long as a condition is true.
- ★ **Condition Checking** happens before each iteration.
- ★ **Loop Control** prevents infinite execution.
- ★ **Initialization** sets starting values for loop variables.

- ★ **Termination Condition** ensures the loop eventually stops.
 - ★ **Example Usage** includes counting, searching, or data processing.
-

Module 91: Pseudocode - Function Structure

- ★ **Function** is a reusable block of code that performs a task.
 - ★ **def Keyword** defines a function in pseudocode.
 - ★ **Parameters** are optional inputs passed to functions.
 - ★ **Function Call** executes the function by its name.
 - ★ **Modularity** breaks complex problems into smaller parts.
 - ★ **Reusability** improves efficiency and clarity of code.
-

Module 92: Discovery - The Art of Problem Solving

- ★ **Problem Solving** involves identifying and resolving issues.
 - ★ **Algorithm Discovery** is about finding new ways to solve tasks.
 - ★ **Step-by-Step Analysis** helps in developing solutions.
 - ★ **Creative Thinking** is essential for novel approaches.
 - ★ **Trial and Error** is a common problem-solving method.
 - ★ **Computer Science** shares strategies with general problem solving.
-

Module 93: Getting Your Foot in the Door

- ★ **Entry Point** refers to starting with a small part of a problem.
 - ★ **Decomposition** means breaking a problem into manageable parts.
 - ★ **Initial Progress** builds confidence for solving bigger issues.
 - ★ **Iteration** allows gradual improvement of the solution.
 - ★ **Simplified Version** is a way to begin solving complex problems.
 - ★ **Foot-in-the-Door Strategy** applies to algorithm design.
-

Module 94: Algorithm Discovery Strategies (I)

- ★ **Guess and Check** tests solutions for correctness.
- ★ **Divide and Conquer** breaks problems into sub-problems.
- ★ **Heuristics** are rules of thumb for faster solutions.
- ★ **Backtracking** undoes steps to try alternatives.
- ★ **Trial Strategy** checks each possibility.
- ★ **Pattern Recognition** helps identify repeated solutions.

Module 95: Algorithm Discovery Strategies (II)

- ★ **Greedy Algorithm** picks the best immediate choice.
 - ★ **Dynamic Programming** saves and reuses previous results.
 - ★ **Brute Force** tests all possibilities exhaustively.
 - ★ **Reduction** transforms one problem into another.
 - ★ **Abstraction** hides complexity for simplicity.
 - ★ **Simulation** tests outcomes of algorithms virtually.
-

Module 96: Iterative Structures - Sequential Search

- ★ **Sequential Search** checks each element one by one.
 - ★ **Linear Search** is another name for sequential search.
 - ★ **Match Check** stops the search upon success.
 - ★ **Not Found** triggers an alternative response.
 - ★ **Looping** is used to iterate through the data.
 - ★ **Simple but Inefficient** for large datasets.
-

Module 97: Loop Control

- ★ **Loop Control** decides when and how loops run.
 - ★ **Initialization** sets starting values.
 - ★ **Condition Check** determines loop continuation.
 - ★ **Update** changes loop variable after each iteration.
 - ★ **Infinite Loop** occurs when the exit condition never meets.
 - ★ **Loop Exit** ends the repetition when criteria are met.
-

Module 98: Components of Repetitive Control

- ★ **Initialization** sets up loop variables.
 - ★ **Condition Test** decides if the loop should proceed.
 - ★ **Body of Loop** contains the repeated instructions.
 - ★ **Update Statement** modifies loop control variable.
 - ★ **Exit Condition** determines the end of the loop.
 - ★ **Iteration** is the process of repeating steps.
-

Module 99: Loop Execution Examples - I

- ★ **Example 1** shows counting loops.
 - ★ **Example 2** demonstrates early exit logic.
 - ★ **Example 3** covers condition-based looping.
 - ★ **Real Use Cases** include sum calculation and searches.
 - ★ **Practical Practice** helps grasp loop mechanics.
 - ★ **Debugging Loops** ensures proper flow and results.
-

Module 100: Loop Execution Examples - II

- ★ **Nested Loops** are loops within loops.
 - ★ **Inner Loop** completes its cycles before the outer continues.
 - ★ **Pattern Printing** uses nested loops.
 - ★ **Table Generation** is another nested loop task.
 - ★ **Tracking Variables** inside loops aids debugging.
 - ★ **Visualizing Output** helps understand loop behavior.
-

LMS Handling Services

Academic Support Services

Get expert help with:

1. Final year projects
2. Internships
3. Management reports
4. Course-specific projects (BNK619, FIN619, HRM619, etc.)
5. Computer Science projects (CS619, CS519)

LMS Handling Services

We also offer assistance with:

- Assignments
- Quizzes
- GDBs (Graded Discussion Boards)
- Lecture watching

Expertise

1. Programming languages: HTML/CSS, PHP, React JS, JavaScript, Java, Python, etc.
2. Personalized guidance
3. Industry-standard solutions

Benefits

1. 100% passing guarantee
2. High passing ratio
3. Report and presentation clearance
4. Cost-effective solutions

Contact

WhatsApp: [+923254755408](https://wa.me/923254755408)

Trustworthy Services

Beware of frauds; rely on authentic services for your academic needs.

Virtual University Of Pakistan

MIDTERM

Exams Alert! +

Struggling to prepare for your midterm exams?

We've got the solution!

Comprehensive Study Material Available:

+ MCQ's File

+ Short Questions File

+ Updated Past Papers

+ Important Topics

Get instant access to our paid notes!

If you're interested,

Contact us: 03254755408

Thank you!